

Contents

GiveSeo Wiki Documentation	3
블록 컴포저 Wiki	3
□ PDF 다운로드	3
블록	3
주석	3
Javascript 코드	4
Python 코드	4
링크	4
Javascript 코드	4
Python 코드	4
종료하기	4
Javascript 코드	5
Python 코드	5
리스트 생성	5
빈 리스트 생성	5
Javascript 코드	5
Python 코드	5
리스트 만들기	5
기본 사용법	5
Javascript 코드	6
Python 코드	6
Javascript 코드	6
Python 코드	6
Javascript 코드	7
Python 코드	7
Javascript 코드	7
Python 코드	7
입력 항목 개수 변경	7
항목으로 리스트 설정	8
Javascript 코드	8
Python 코드	9
리스트 길이	9
비어 있습니다	9
Javascript 코드	9
Python 코드	9
길이	9
Javascript 코드	10
Python 코드	10
Javascript 코드	10
Python 코드	10
리스트 나타난 위치	11
Javascript 코드	11
Python 코드	12
리스트에서 항목 가져오기	12
단일 항목 가져오기	12
Javascript 코드	13
Python 코드	14

항목 가져오기 및 제거	14
Javascript 코드	14
Python 코드	15
Javascript 코드	15
Python 코드	16
항목 제거	16
Javascript 코드	16
Python 코드	17
서브리스트 추출	17
Javascript 코드	17
Python 코드	18
Javascript 코드	18
Python 코드	18
리스트에 항목 추가하기	19
리스트에서 ... 설정	19
Javascript 코드	19
Python 코드	19
Javascript 코드	20
Python 코드	20
리스트에서 ... 원하는 위치에 삽입	21
Javascript 코드	21
Python 코드	21
Javascript 코드	22
Python 코드	22
리스트 값 치환 및 리스트 반환	22
Javascript 코드	22
Python 코드	23
Javascript 코드	23
Python 코드	24
문자열 분할 및 리스트 결합	24
텍스트에서 목록 만들기	24
Javascript 코드	24
Python 코드	24
목록에서 텍스트 만들기	24
Javascript 코드	24
Python 코드	25
관련 블록	25
리스트 출력하기	25
Javascript 코드	25
Python 코드	25
리스트의 각 항목에 대해	25
Javascript 코드	26
Python 코드	26
리스트 순서	26
정렬	26
Javascript 코드	27
Python 코드	28
Javascript 코드	29
Python 코드	29

뒤집기	30
Javascript 코드	30
Python 코드	30
Javascript 코드	30
Python 코드	31
드롭다운 메뉴	32
블록	32
설정	32
Javascript 코드	32
Python 코드	33
값 가져오기	33
Javascript 코드	33
Python 코드	33
바꾸기	33
Javascript 코드	34
Python 코드	34
예제	34
Javascript 코드	34
Python 코드	34

GiveSeo Wiki Documentation

블록 컴포저 Wiki

블록 컴포저는 다양한 기본 블록을 제공합니다. 이 블록들은 다음과 같은 카테고리로 나누어집니다:

□ PDF 다운로드

[□ 최신 Wiki PDF 다운로드](#)

블록 코딩에서 **기타 블록**은 코드 실행 및 로봇의 동작에 영향을 주지 않는 코드들로 구성됩니다. 주석을 달거나, 코드 실행을 종료하는 기능을 수행할 수 있습니다.

블록

주석

주석 블록을 사용하면 코드의 실행에는 영향을 주지 않으면서 **설명**을 추가할 수 있습니다. 주석을 활용하면 코드 가독성이 높아지고, 유지보수가 쉬워집니다.

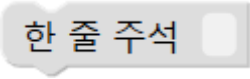
A light gray rounded rectangle with a speech bubble tail pointing to the left. Inside, the text "한 줄 주석" is in a dark blue font, followed by a small white square button.

Figure 1: Image

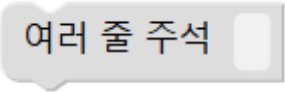
A light gray rounded rectangle with a speech bubble tail pointing to the left. Inside, the text "여러 줄 주석" is in a dark blue font, followed by a small white square button.

Figure 2: Image

Javascript 코드

```
// 한 줄 주석  
  
//여러 줄 주석  
//이어서 입력
```

Python 코드

```
# 한 줄 주석  
  
# 여러 줄 주석  
# 이어서 입력
```

링크

링크 블록을 사용하면, 주석 기능을 활용해 열고 싶은 페이지 링크를 추가할 수 있습니다.
열기 버튼을 클릭하면, 입력한 링크의 페이지로 이동할 수 있습니다.

Javascript 코드

```
// https://google.com
```

Python 코드

```
# https://google.com
```

종료하기

종료하기 블록은 프로그램 실행을 즉시 중단하고 페이지를 새로고침하여 초기 상태로 되돌립니다. 특정 조건에서 강제 종료 기능을 추가할 때 유용합니다.

실행 종료하기

Figure 3: Image

Javascript 코드

```
__exit(); // 종료하기
```

Python 코드

```
__exit() # 종료하기
```

일상적인 말처럼, 블록 컴포저에서 리스트는 “할 일 목록”이나 “쇼핑 목록”처럼 항목들이 순서대로 나열된 집합입니다. 리스트의 항목들은 어떤 타입이라도 될 수 있으며, 동일한 값이 리스트에 여러 번 나타날 수 있습니다.

리스트 생성

빈 리스트 생성

가장 간단한 리스트는 빈 리스트로, 빈 리스트 생성 블록을 사용하여 생성합니다:

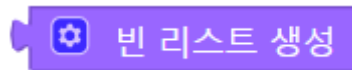


Figure 4: Image

Javascript 코드

```
[]; // 빈 리스트
```

Python 코드

```
[] # 빈 리스트
```

리스트 만들기

기본 사용법

리스트 만들기 블록을 사용하면 새로운 리스트에 초기값을 지정할 수 있습니다. 예를 들어, 단어 리스트를 만들고 이를 **letters**라는 변수에 저장할 수 있습니다:

이 문서에서는 이 리스트를 [“하나”, “둘”, “셋”]으로 나타냅니다. 이 블록에서 정의된 변수들은 이후 예제에서 사용될 것입니다.

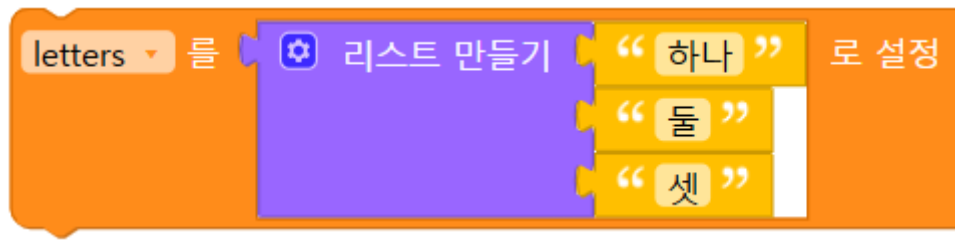


Figure 5: Image

JavaScript 코드

```
var letters;

letters = ['하나', '둘', '셋'];
```

Python 코드

```
letters = None

letters = ['하나', '둘', '셋']
```

다음은 숫자 형태의 문자열 리스트를 만드는 예입니다:

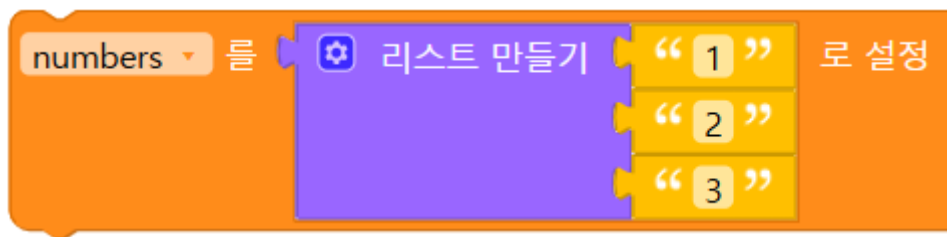


Figure 6: Image

JavaScript 코드

```
var numbers;

numbers = ['1', '2', '3'];
```

Python 코드

```
numbers = None

numbers = ['1', '2', '3']
```

다음은 색상 리스트를 만드는 예입니다:

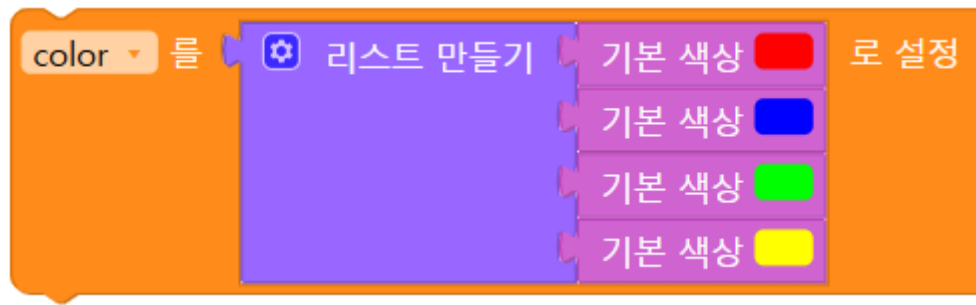


Figure 7: Image

JavaScript 코드

```
color = [[255, 0, 0], [0, 0, 255], [0, 255, 0], [255, 255, 0]];
```

Python 코드

```
color = [[255, 0, 0], [0, 0, 255], [0, 255, 0], [255, 255, 0]]
```

덜 흔하지만, 다양한 타입의 값을 가진 리스트도 만들 수 있습니다:



Figure 8: Image

JavaScript 코드

```
['하나', 1, [255, 0, 0]];
```

Python 코드

```
['하나', 1, [255, 0, 0]]
```

입력 항목 개수 변경

입력 항목의 개수를 변경하려면 기어 아이콘을 클릭하세요. 새로운 창이 열리며, 여기서 왼쪽의 항목 서브 블록을 오른쪽의 리스트 블록으로 드래그하여 새로운 입력을 추가할 수 있습니다:

새 항목은 원하는 위치에 추가할 수 있습니다. 마찬가지로, 원하지 않는 항목 서브 블록은 왼쪽으로 드래그하여 제거할 수 있습니다.

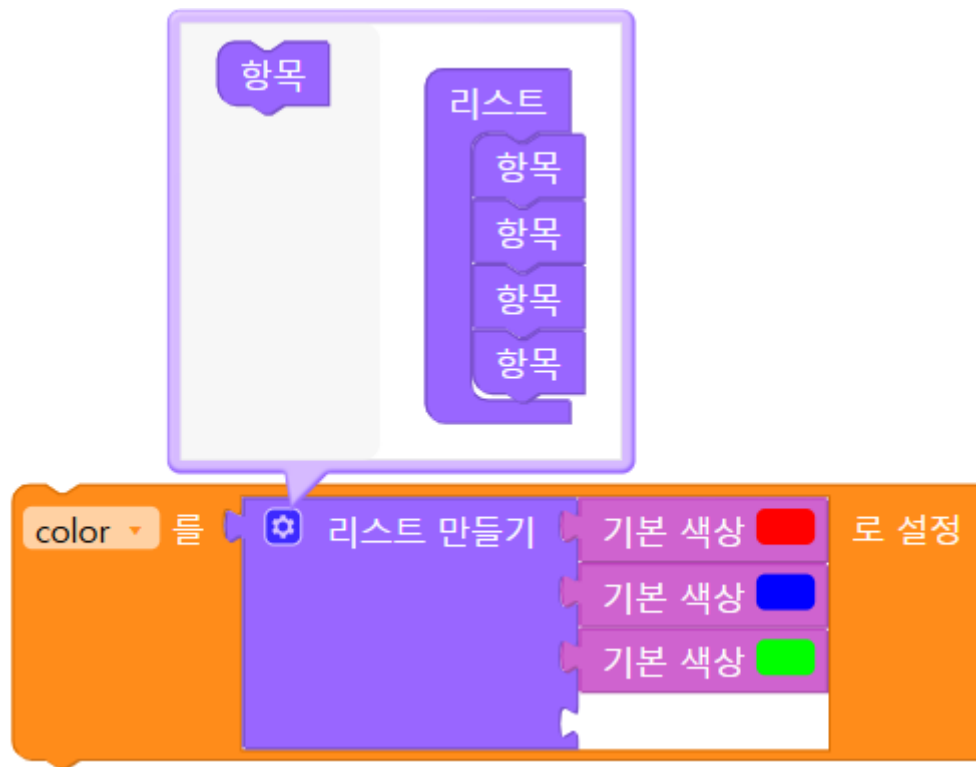


Figure 9: Image

항목으로 리스트 설정

항목으로 리스트 설정 블록을 사용하면 지정된 항목을 반복하여 원하는 개수만큼 리스트를 만들 수 있습니다. 예를 들어, 아래의 블록은 변수 **words**를 ["very", "very", "very"]로 설정합니다.



Figure 10: Image

Javascript 코드

```
var words;

function listsRepeat(value, n) { // 리스트 내 동일 값 반복 생성 함수
  var array = [];
  for (var i = 0; i < n; i++) {
    array[i] = value;
  }
  return array;
}

words = listsRepeat('very', 3); // words = ['very', 'very', 'very']
```


Python 코드

```
words = None
words = ['very'] * 3 # words = ['very', 'very', 'very']
```

리스트 길이

비어 있습니다

비어 있습니다 블록의 값은 입력이 빈 리스트일 경우 참, 그 외에는 거짓입니다(리스트가 아니더라도 마찬가지). 아래 블록의 값은 거짓이 됩니다. 왜냐하면 **color** 변수는 비어 있지 않으며, 3개의 항목을 가지고 있기 때문입니다.

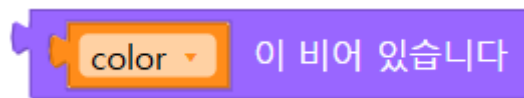


Figure 11: Image

Javascript 코드

```
var color;

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0]];
!color.length; // 비어있지 않으므로 거짓 출력
```

Python 코드

```
color = None

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0]]
not len(color) # 비어있지 않으므로 거짓 출력
```

이것은 문자열의 “빈 문자열 확인” 블록과 유사합니다.

길이

길이 블록의 값은 리스트의 항목 수입니다. 예를 들어, 아래 블록에서 **color**는 3개의 항목을 가졌기 때문에 값은 3입니다.

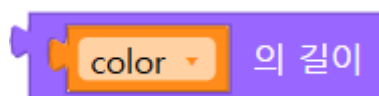


Figure 12: Image

JavaScript 코드

```
var color;

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0]];
color.length; // 3개의 항목이므로 3의 값 반환
```

Python 코드

```
color = None

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0]]
len(color) # 3개의 항목이므로 3의 값 반환
```

길이 블록은 리스트에 몇 개의 항목이 있는지 알려줍니다. 리스트 내에 다른 항목이 몇 개 있는지가 아니라, 그 자체의 항목 수를 세는 것입니다. 예를 들어, 아래는 값이 3인데, **words**는 세 번 반복된 동일한 텍스트(["very", "very", "very"])로 구성되어 있습니다.



Figure 13: Image

이것은 문자열의 “문자열 길이” 블록과 유사합니다.

JavaScript 코드

```
var words;

function listsRepeat(value, n) { // 리스트 내 동일 값 반복 생성 함수
  var array = [];
  for (var i = 0; i < n; i++) {
    array[i] = value;
  }
  return array;
}

words = listsRepeat('very', 3);
words.length; // 3개의 항목이므로 3의 값 반환
```

Python 코드

```
words = None

words = ['very'] * 3
len(words) # 3개의 항목이므로 3의 값 반환
```

리스트 나타난 위치

이 블록들은 리스트에서 항목의 위치를 찾습니다. 예를 들어, 아래 블록의 값은 1입니다. 이유는 “very”라는 첫 번째 항목이 **words** 리스트(["very", "very", "very"])에서 첫 번째 위치에 있기 때문입니다.

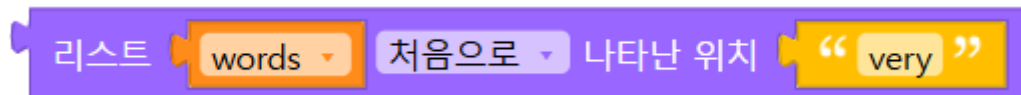


Figure 14: Image

다음 결과는 3입니다. 왜냐하면 “very”라는 항목이 **words** 리스트에서 마지막 위치에 있기 때문입니다.

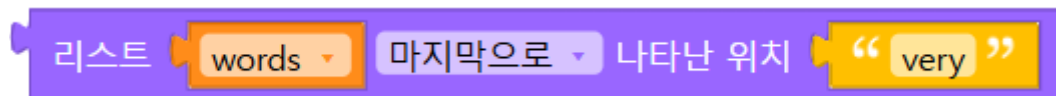


Figure 15: Image

리스트에 항목이 없는 경우, 결과는 0입니다. 예를 들어:

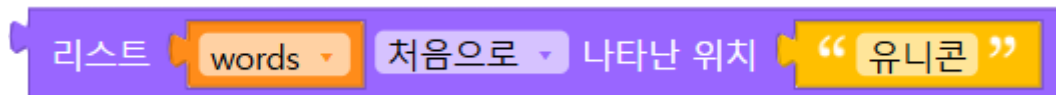


Figure 16: Image

이 블록들은 문자열의 “문자열 찾기” 블록과 유사합니다.

Javascript 코드

```
var words;

function listsRepeat(value, n) { // 리스트 내 동일 값 반복 생성 함수
  var array = [];
  for (var i = 0; i < n; i++) {
    array[i] = value;
  }
  return array;
}

words = listsRepeat('very', 3);
```

```
words.indexOf('very') + 1 // "very" 문자열이 첫 번째로 나타난 위치 1 반환
words.lastIndexOf('very') + 1 // "very" 문자열이 마지막으로 나타난 위치 3 반환
words.indexOf('유니콘') + 1 // "유니콘" 문자열이 첫 번째로 나타난 위치 0 반환
```

Python 코드

```
words = None

def first_index(my_list, elem): # 문자열 첫 번째 위치 찾기 함수
    try: index = my_list.index(elem) + 1
    except: index = 0
    return index

def last_index(my_list, elem): # 문자열 마지막 위치 찾기 함수
    try: index = len(my_list) - my_list[::-1].index(elem)
    except: index = 0
    return index

words = ['very'] * 3
first_index(words, 'very') # "very" 문자열이 첫 번째로 나타난 위치 1 반환
last_index(words, 'very') # "very" 문자열이 마지막으로 나타난 위치 3 반환
first_index(words, '유니콘') # "유니콘" 문자열이 첫 번째로 나타난 위치 0 반환
```

리스트에서 항목 가져오기

단일 항목 가져오기

color 리스트의 정의를 기억하세요:

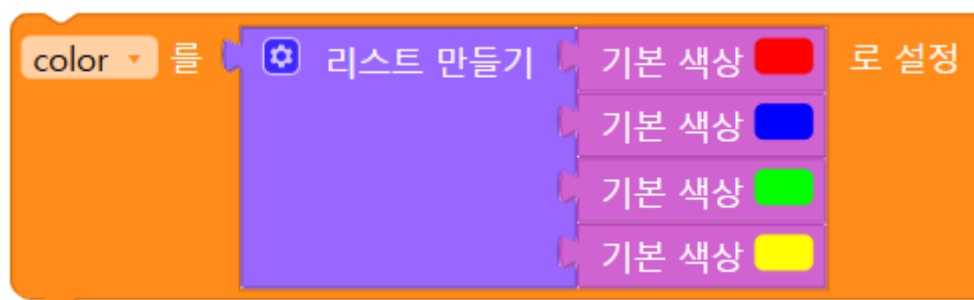


Figure 17: Image

다음 블록은 색상 파랑을 가져옵니다. 파랑은 리스트의 두 번째 항목이기 때문입니다(왼쪽에서부터 세기):

이 블록은 초록을 가져옵니다. 초록은 오른쪽 끝에서 두 번째 항목이기 때문입니다:



Figure 18: Image

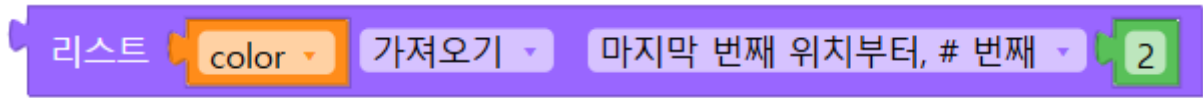


Figure 19: Image

이 블록은 첫 번째 항목인 빨강을 가져옵니다:



Figure 20: Image

이 블록은 마지막 항목인 노랑을 가져옵니다:

이 블록은 리스트에서 임의로 하나의 항목을 선택합니다. 빨강, 파랑, 초록, 노랑 중 하나를 동일한 확률로 선택합니다.

Javascript 코드

```
var color;
```

```
function listsGetRandomItem(list, remove) { // 리스트 무작위 항목 선택 함수
  var x = Math.floor(Math.random() * list.length);
  if (remove) {
    return list.splice(x, 1)[0];
  } else {
    return list[x];
  }
}
```

```
color = [[255, 0, 0], [0, 0, 255], [0, 255, 0], [255, 255, 0]];
color[1]; // color에서 두 번째 항목 파랑 가져오기
color.slice(-2)[0]; // color에서 끝에서 두 번째 항목 초록 가져오기
color[0]; // color에서 첫 번째 항목 빨강 가져오기
color.slice(-1)[0]; // color에서 마지막 항목 노랑 가져오기
listsGetRandomItem(color, false); // color에서 무작위 항목 가져오기
```



Figure 21: Image



Figure 22: Image

Python 코드

```
import random

color = None

color = [[255, 0, 0], [0, 0, 255], [0, 255, 0], [255, 255, 0]]
color[1] # color에서 두 번째 항목 파랑 가져오기
color[-2] # color에서 끝에서 두 번째 항목 초록 가져오기
color[0] # color에서 첫 번째 항목 빨강 가져오기
color[-1] # color에서 마지막 항목 노랑 가져오기
random.choice(color) # color에서 무작위 항목 가져오기
```

항목 가져오기 및 제거

리스트에서 ... 가져오기 블록에서 드롭다운 메뉴를 선택하면 리스트에서 ... 가져오기 및 제거로 변경되어, 항목을 가져오는 것뿐만 아니라 원본 리스트도 수정합니다:

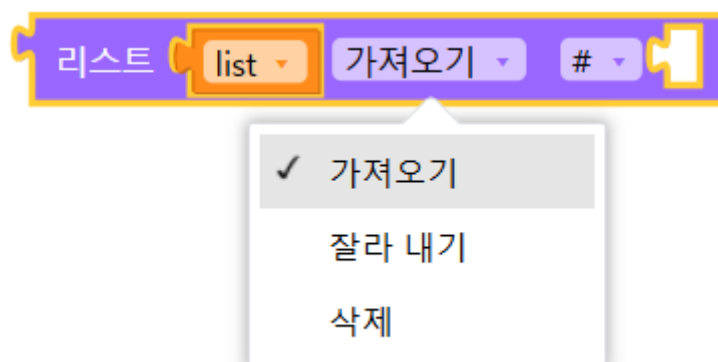


Figure 23: Image

Javascript 코드

```
var list;
```

```
function listsGetRandomItem(list, remove) { // 리스트 랜덤 항목 추출 함수
  var x = Math.floor(Math.random() * list.length);
  if (remove) {
    return list.splice(x, 1)[0];
  } else {
    return list[x];
  }
}
```

```
list[0]; // 리스트 첫 번째 항목 가져오기
list.slice(-1)[0]; // 리스트 마지막 항목 가져오기
listsGetRandomItem(list, false); // 리스트 랜덤 항목 가져오기
list.splice(0, 1)[0]; // 리스트 첫 번째 항목 잘라내기
list.shift(); // 리스트 앞에서 항목 제거 (첫 번째 항목 반환)
list.pop(); // 리스트 마지막 항목 잘라내기
listsGetRandomItem(list, true); // 리스트 임의 항목 잘라내기
```

Python 코드

```
list2 = None # list는 예약어 이기 때문에 list2 사용

def lists_remove_random_item(myList): # 리스트 임의 잘라내기 함수
  x = int(random.random() * len(myList))
  return myList.pop(x)

list2[0] # 리스트 첫 번째 항목 가져오기
list2[-1] # 리스트 마지막 항목 가져오기
random.choice(list2) # 리스트 랜덤 항목 가져오기
list2.pop(0) # 리스트 첫 번째 항목 잘라내기
list2.pop(0) # 리스트 앞에서 항목 제거 (첫 번째 항목 반환)
list2.pop() # 리스트 마지막 항목 잘라내기
lists_remove_random_item(list2) # 리스트 임의 항목 잘라내기
```

이 예제는 변수 **first letter**에 ["하나"]를 설정하고, **letters**는 ["둘", "셋"]으로 남겨둡니다.

Javascript 코드

```
var letters, first_letter;

letters = ['하나', '둘', '셋'];
```

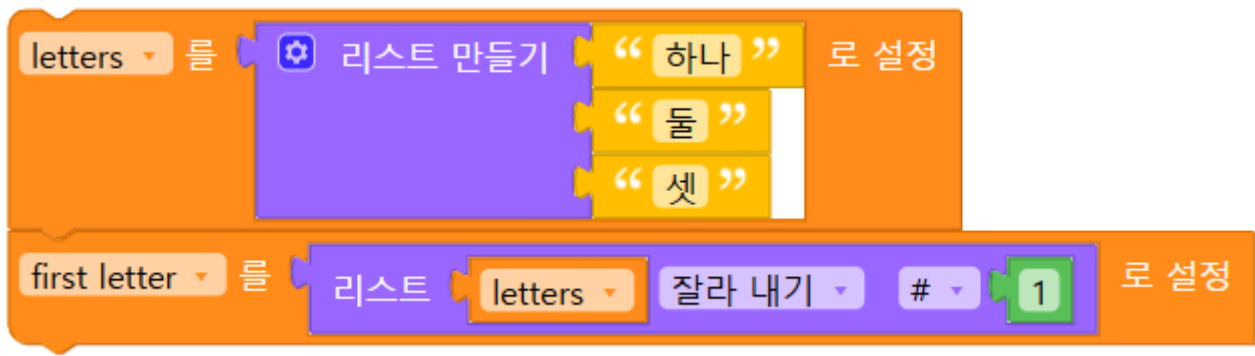


Figure 24: Image

```
first_letter = letters.splice(0, 1)[0]; // letters 리스트 첫 번째 항목 잘라내기 first_letter = ["하나"], letters = ["둘", "셋"]
```

Python 코드

```
letters = None
first_letter = None

letters = ['하나', '둘', '셋']
first_letter = letters.pop(0) # letters 리스트 첫 번째 항목 잘라내기 first_letter = ["하나"], letters = ["둘", "셋"]
```

항목 제거

드롭다운에서 “삭제”를 선택하면 블록 왼쪽의 플러그가 사라집니다:

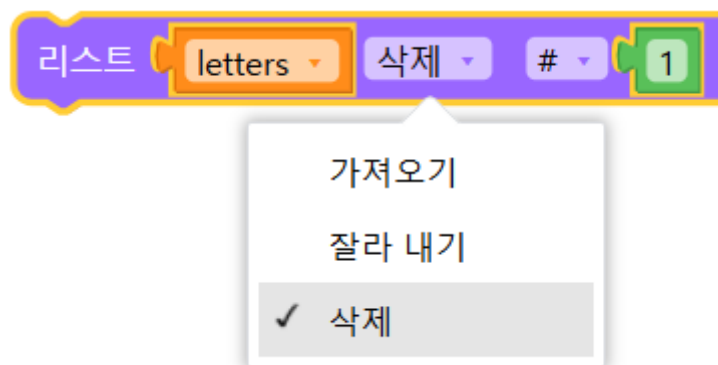


Figure 25: Image

이렇게 하면 **letters**에서 첫 번째 항목이 제거됩니다.

Javascript 코드

```
var letters;

letters = ['하나', '둘', '셋'];
```



```
letters.splice(0, 1); // letters 리스트 첫 번째 항목 삭제, letters = ["둘", "셋"]
```

Python 코드

```
letters = None
```

```
letters = ['하나', '둘', '셋']
```

```
letters.pop(0) # letters 리스트 첫 번째 항목 삭제, letters = ["둘", "셋"]
```

서브리스트 추출

리스트에서 ... 서브리스트 추출 블록은 리스트에서 ... 가져오기 블록과 유사하지만, 개별 항목 대신 서브리스트를 추출합니다. 서브리스트의 시작과 끝을 지정하는 방법은 여러 가지가 있습니다:

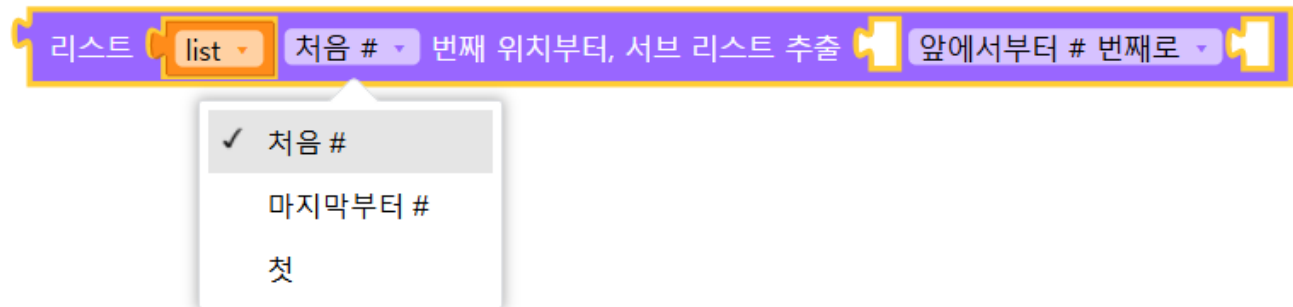


Figure 26: Image

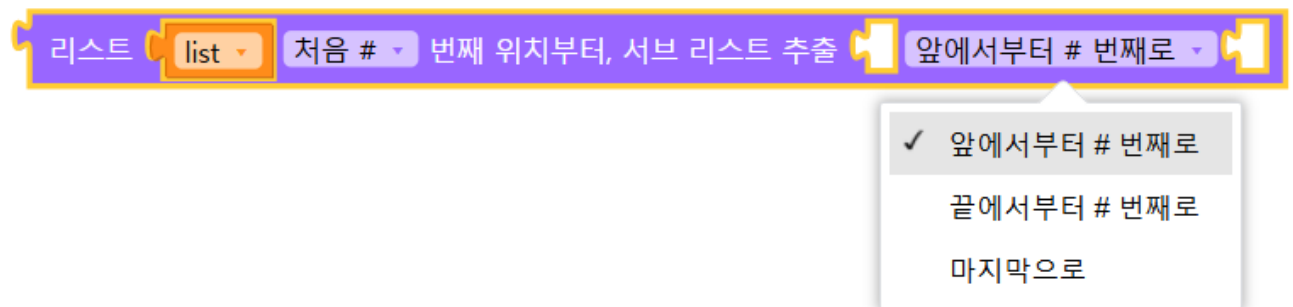


Figure 27: Image

Javascript 코드

```
var list;
```

```
list.slice(0, 1); // 첫 번째 위치부터 첫 번째 항목까지 서브리스트 추출
```

```
list.slice(0, list.length - 0); // 첫 번째 위치부터 끝에서 첫 번째까지 서브리스트 추출
```

```
list.slice(0, list.length); // 첫 번째 위치부터 마지막까지 서브리스트 추출
```

```
list.slice(list.length - 1, 1); // 마지막 첫 번째 위치부터 마지막까지 서브리스트 추출
```

```
list.slice(list.length - 1, list.length - 0); // 마지막 첫 번째 위치부터 마지막까지 서브리스트 추출
```

```
list.slice(list.length - 1, list.length); // 마지막 첫 번째 위치부터 마지막까지 서브리스트 추출
list.slice(0, 1); // 첫 번째 위치부터 첫 번째 항목까지 서브리스트 추출
list.slice(0, list.length - 0); // 첫 번째 위치부터 끝까지 서브리스트 추출
list.slice(0); // 첫 번째 위치부터 마지막까지 서브리스트 추출
```

Python 코드

```
list2 = None # list는 예약어 이기 때문에 list2 사용

list2[:1] # 첫 번째 위치부터 첫 번째 항목까지 서브리스트 추출
list2[: ] # 첫 번째 위치부터 끝까지 서브리스트 추출
list2[: ] # 첫 번째 위치부터 마지막까지 서브리스트 추출
list2[-1:1] # 마지막 첫 번째 위치부터 끝까지 서브리스트 추출
list2[-1:] # 마지막 첫 번째 위치부터 끝까지 서브리스트 추출
list2[-1:] # 마지막 첫 번째 위치부터 마지막까지 서브리스트 추출
list2[:1] # 첫 번째 위치부터 첫 번째 항목까지 서브리스트 추출
list2[: ] # 첫 번째 위치부터 끝까지 서브리스트 추출
list2[: ] # 첫 번째 위치부터 마지막까지 서브리스트 추출
```

이 예제에서는 새로운 리스트 **first letters**가 생성됩니다. 이 리스트에는 두 개의 항목이 포함됩니다: ["하나", "둘"].

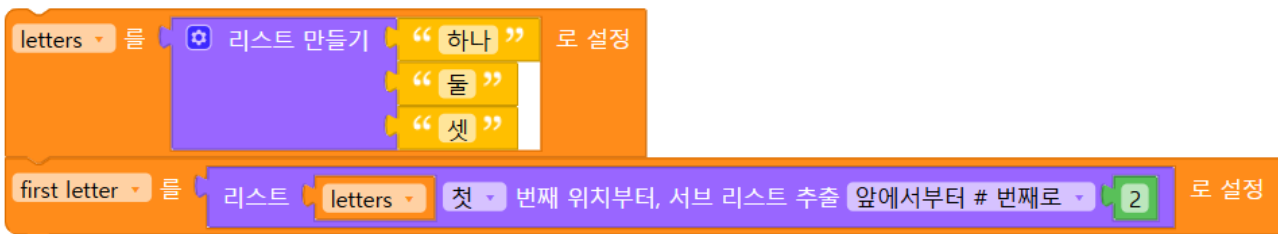


Figure 28: Image

이 블록은 원본 리스트를 수정하지 않음을 유의하세요.

Javascript 코드

```
var letters, first_letter, list;

letters = ['하나', '둘', '셋'];
first_letter = list.slice(0, 2); // 첫 번째 위치부터 두 번째 항목까지 추출 first_letter = ["하나", "둘"]
```

Python 코드

```
letters = None
first_letter = None
```

```
list2 = None # list는 예약어 이기 때문에 list2 사용

letters = ['하나', '둘', '셋']
first_letter = list2[:2] # 첫 번째 위치부터 두 번째 항목까지 추출 first_letter = ["하나", "둘"]
```

리스트에 항목 추가하기

리스트에서 ... 설정

리스트에서 ... 설정 블록은 지정된 위치에 있는 항목을 다른 항목으로 교체합니다.

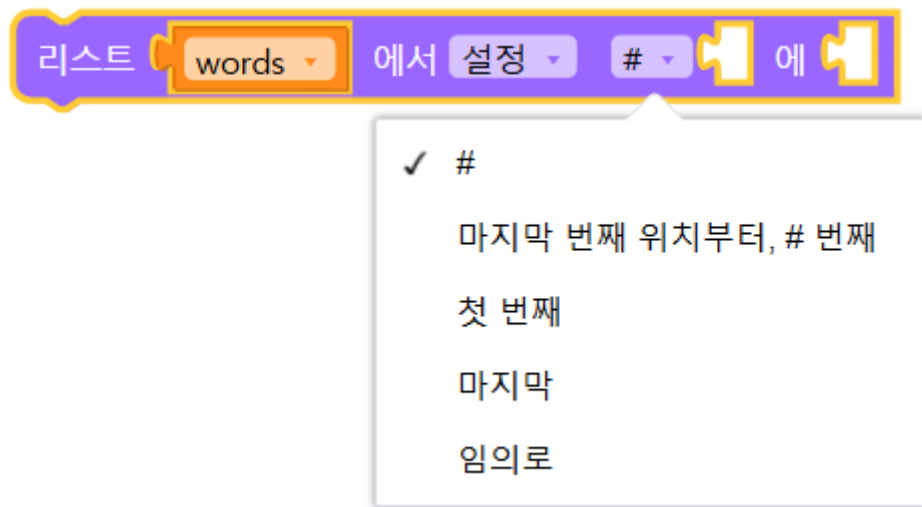


Figure 29: Image

Javascript 코드

```
var letters, list;

list[0] = 'hello'; // 리스트에 첫 번째 위치에 "hello" 문자열 추가하기
list[list.length - 1] = 'hello'; // 리스트에 마지막 위치부터 "hello" 문자열 추가하기
list[0] = 'hello'; // 리스트 첫 번째에 "hello" 문자열 추가하기
list[list.length - 1] = 'hello'; // 리스트에 마지막 위치에 "hello" 문자열 추가하기
var tmpX = Math.floor(Math.random() * list.length); // 무작위 위치 계산
list[tmpX] = 'hello'; // 리스트에 무작위 위치에 "hello" 문자열 추가하기
```

Python 코드

```
import random

letters = None
list2 = None # list는 예약어 이기 때문에 list2 사용
```

```
list2[0] = 'hello' # 리스트에 첫 번째 위치에 "hello" 문자열 추가하기
list2[-1] = 'hello' # 리스트에 마지막 위치부터 "hello" 문자열 추가하기
list2[0] = 'hello' # 리스트 첫 번째에 "hello" 문자열 추가하기
list2[-1] = 'hello' # 리스트의 마지막 위치에 "hello" 문자열 추가하기
tmp_x = int(random.random() * len(list2)) # 무작위 위치 계산
list2[tmp_x] = 'hello' # 리스트에 무작위 위치에 "hello" 문자열 추가하기
```

드롭다운 옵션의 의미는 **이전 섹션**을 참조하세요.

다음 예제는 두 가지 일을 합니다: 1. **words** 리스트를 ["very", "very", "very"]로 생성합니다. 2. 리스트의 세 번째 항목을 "good"으로 교체합니다. 새 값은 **words**가 ["very", "very", "good"]이 됩니다.



Figure 30: Image

JavaScript 코드

```
var words;

function listsRepeat(value, n) { // 리스트 내 동일 값 반복 생성 함수
  var array = [];
  for (var i = 0; i < n; i++) {
    array[i] = value;
  }
  return array;
}

words = listsRepeat('very', 3); // words = ["very", "very", "very"]
words[2] = 'good'; // words = ["very", "very", "good"]
```

Python 코드

```
words = None

words = ['very'] * 3 # words = ['very', 'very', 'very']
words[2] = 'good' # words = ['very', 'very', 'good']
```

리스트에서 ... 원하는 위치에 삽입

리스트에서 ... 원하는 위치에 삽입 블록은 리스트에서 ... 설정 블록에서 드롭다운 메뉴를 사용하여 얻을 수 있습니다:

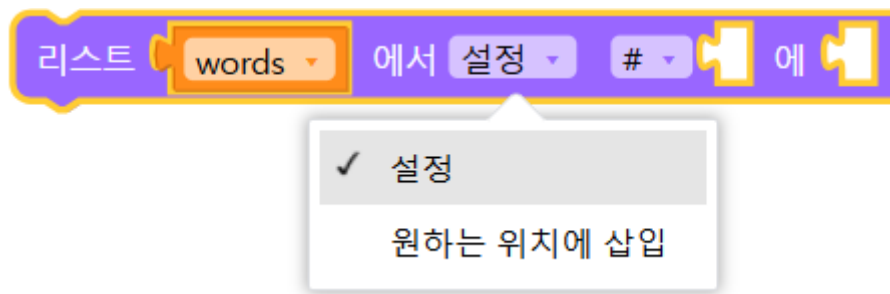


Figure 31: Image

Javascript 코드

```
var words;

words.splice(2, 0, 'good'); // 세 번째 위치 앞에 "good" 문자열 삽입
words.splice(words.length - 3, 0, 'good'); // 마지막으로부터 3번째 위치 앞에 "good" 문자열 삽입
words.unshift('good'); // 맨 앞에 "good" 문자열 삽입
words.push('good'); // 마지막 위치에 "good" 문자열 삽입
var tmpX = Math.floor(Math.random() * words.length); // 무작위 위치 계산
words.splice(tmpX, 0, 'good'); // 무작위 위치에 "good" 문자열 삽입
```

Python 코드

```
import random

words = None

words.insert(2, 'good') # 세 번째 위치 앞에 "good" 문자열 삽입
words.insert(-3, 'good') # 마지막으로부터 3번째 위치 앞에 "good" 문자열 삽입
words.insert(0, 'good') # 맨 앞에 "good" 문자열 삽입
words.append('good') # 마지막 위치에 "good" 문자열 삽입
tmp_x = int(random.random() * len(words)) # 무작위 위치 계산
words.insert(tmp_x, 'good') # 무작위 위치에 "good" 문자열 삽입
```

이 블록은 새 항목을 리스트의 지정된 위치에 삽입합니다. 예를 들어, 아래 예제에서는 세 가지 일이 일어납니다: 1. **words** 리스트는 ["very", "very", "very"]로 생성됩니다. 2. 리스트의 세 번째 항목이 "good"으로 교체됩니다. 새로운 값은 ["very", "very", "good"]입니다. 3. "you're"라는 단어가 리스트의 맨 앞에 삽입됩니다. 최종 값은 ["You're", "very", "very", "good"]입니다.

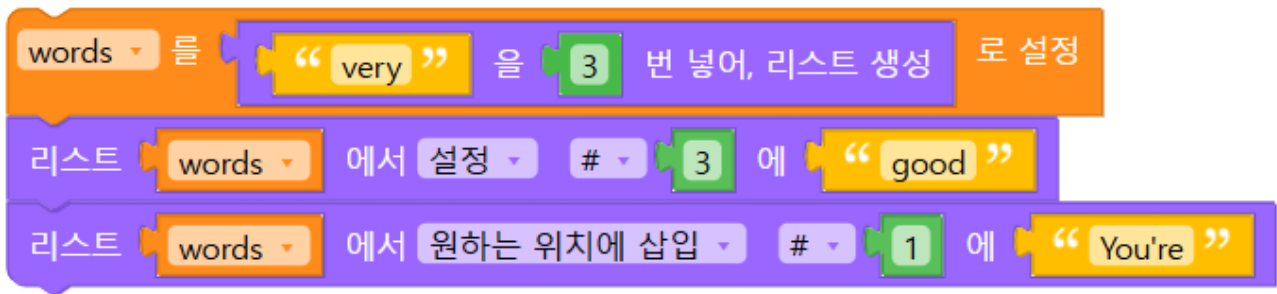


Figure 32: Image

Javascript 코드

```
var words;

function listsRepeat(value, n) { // 리스트 내 동일 값 반복 생성 함수
  var array = [];
  for (var i = 0; i < n; i++) {
    array[i] = value;
  }
  return array;
}

words = listsRepeat('very', 3);
words[2] = 'good'; // words = ["very", "very", "good"]
words.splice(0, 0, 'You're'); // words = ["You're", "very", "very", "good"]
```

Python 코드

```
words = None

words = ['very'] * 3
words[2] = 'good' # words = ['very', 'very', 'good']
words.insert(0, "You're") # words = ['You're', 'very', 'very', 'good']
```

리스트 값 치환 및 리스트 반환

리스트 ... 항목을 ... (으)로 바꾼 리스트 추출 블록은 지정된 위치에 있는 항목을 다른 항목으로 치환한 뒤, 전체 리스트를 반환합니다.

Javascript 코드

```
var words;
```

```
words.map((data, idx) => (idx === 0 ? 'good' : data)) // words 리스트의 첫번째 항목을 "good"으로 바꾼 리스트 추출  
words.map((data, idx) => (idx === words.length - 1 ? 'good' : data)) // words 리스트의 마지막으로부터 첫번째 항목  
words.map((data, idx) => (idx === 0 ? 'good' : data)) // words 리스트의 첫번째 항목을 "good"으로 바꾼 리스트 추출  
words.map((data, idx) => (idx === words.length - 1 ? 'good' : data)) // words 리스트의 마지막 항목을 "good"으로 바꾼 리스트 추출
```

Python 코드

```
words = None
```

```
['good' if i == 0 else data for i, data in enumerate(words)] # words 리스트의 첫번째 항목을 "good"으로 바꾼 리스트 생성  
['good' if i == len(words) - 1 else data for i, data in enumerate(words)] # words 리스트의 마지막으로부터 첫번째 항목  
['good' if i == 0 else data for i, data in enumerate(words)] # words 리스트의 첫번째 항목을 "good"으로 바꾼 리스트 생성  
['good' if i == len(words) - 1 else data for i, data in enumerate(words)] # words 리스트의 마지막 항목을 "good"으로 바꾼 리스트 생성
```

드롭다운 옵션의 의미는 [이전 섹션](#)을 참조하세요.

다음 예제는 세 가지 일을 합니다:

1. **words** 리스트를 ["very", "very", "very"]로 생성합니다.
2. 리스트의 세 번째 항목을 "good"으로 교체한 새로운 리스트를 반환하고, 해당 리스트를 새로운 변수 **list**에 할당합니다.
3. **list** 리스트는 ["very", "very", "good"]이 됩니다. **words** 리스트의 값은 바뀌지 않습니다.

Javascript 코드

```
var words, list;
```

```
function listsRepeat(value, n) { // 리스트 내 동일 값 반복 생성 함수  
  var array = [];  
  for (var i = 0; i < n; i++) {  
    array[i] = value;  
  }  
  return array;  
}
```

```
words = listsRepeat('very', 3); // words = ["very", "very", "very"]  
list = (list.map((data, idx) => (idx === 2 ? 'good' : data))); // list = ["very", "very", "good"]
```

Python 코드

```
words = None
list2 = None # list는 예약어 이기 때문에 list2 사용

words = ['very'] * 3 # words = ['very', 'very', 'very']
list2 = ['good' if i == 2 else data for i, data in enumerate(list2)] # list2 = ['very', 'very', 'good']
```

문자열 분할 및 리스트 결합

텍스트에서 목록 만들기

텍스트에서 목록 만들기 블록은 주어진 텍스트를 구분자를 사용하여 조각으로 나눕니다:

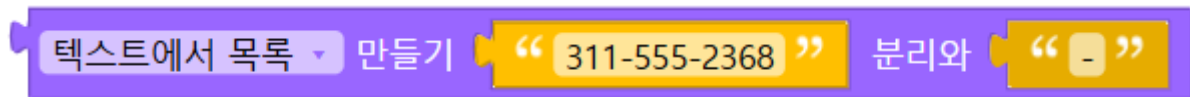


Figure 33: Image

Javascript 코드

```
'311-555-2368'.split('-'); // '-' 를 기준으로 분리, ["311", "555", "2368"]
```

Python 코드

```
'311-555-2368'.split('-') # '-' 를 기준으로 분리, ["311", "555", "2368"]
```

위 예제에서 새 리스트는 “311”, “555”, “2368”로 나눕니다.

목록에서 텍스트 만들기

목록에서 텍스트 만들기 블록은 구분자를 사용하여 리스트의 항목들을 하나의 텍스트로 결합합니다:

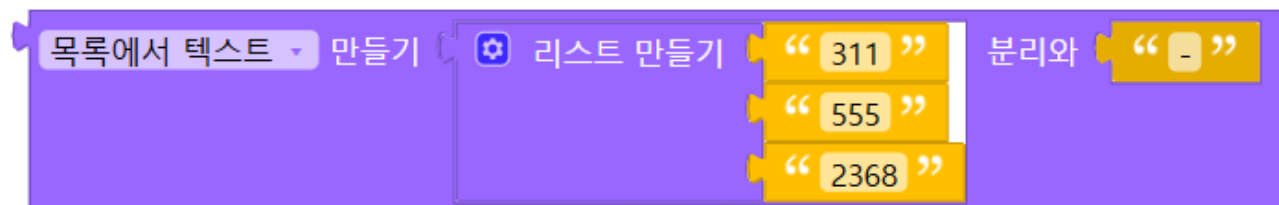


Figure 34: Image

Javascript 코드


```
[ '311', '555', '2368' ].join('-'); // '-'를 구분자로 사용하여 리스트를 하나의 문자열로 결합 "311-555-2368"
```

Python 코드

```
'-'.join(['311', '555', '2368']) # '-'를 구분자로 사용하여 리스트를 하나의 문자열로 결합 "311-555-2368"
```

위 예제에서 새 텍스트는 “311-555-2368”로 반환됩니다.

관련 블록

리스트 출력하기

텍스트의 “출력” 블록은 리스트를 출력할 수 있습니다. 아래 프로그램의 결과는 알림 상자에 출력됩니다:



Figure 35: Image

Javascript 코드

```
var letters;  
  
letters = ['하나', '둘', '셋'];  
window.alert(letters); // 리스트 출력하기
```

Python 코드

```
letters = None  
  
letters = ['하나', '둘', '셋']  
print(letters) # 리스트 출력하기
```

하나,둘,셋이 출력됩니다.

리스트의 각 항목에 대해

반복의 “각 항목에 대해” 블록은 리스트의 각 항목에 대해 연산을 수행합니다. 예를 들어, 아래 블록들은

리스트의 각 항목을 개별적으로 출력합니다:

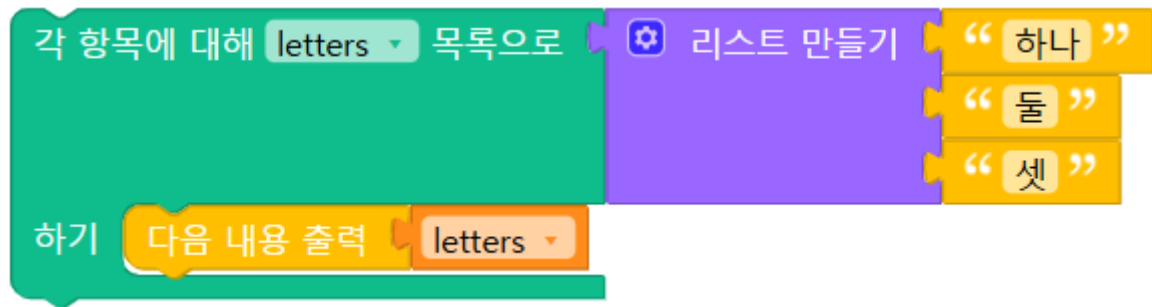


Figure 36: Image

Javascript 코드

```
var letters;

var letters_list = ['하나', '둘', '셋'];
for (var letters_index in letters_list) { // 리스트 각 항목에 대해 출력하기
  letters = letters_list[letters_index];
  window.alert(letters);
}
```

Python 코드

```
letters = None
for letters in ['하나', '둘', '셋']: # 리스트 각 항목에 대해 출력하기
  print(letters)
```

하나, 둘, 셋이 순차적으로 출력됩니다. 이 작업은 원본 리스트에서 항목을 제거하지 않습니다.

반복 종료 블록들의 예시도 참고하세요.

리스트 순서

리스트 항목의 순서를 변경할 수 있는 블록입니다.

정렬

리스트를 원하는 기준에 따라 정렬하는 블록입니다. 숫자 또는 알파벳을 기준으로 정렬할 수 있으며, 오름차순과 내림차순을 선택할 수 있습니다. 또한, 알파벳 정렬 시 대소문자를 무시하고 정렬할 수도 있습니다.

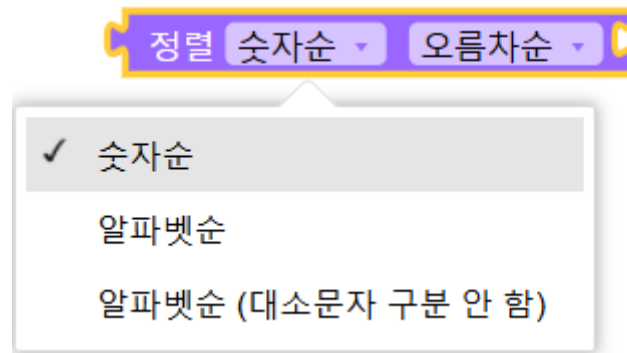


Figure 37: Image

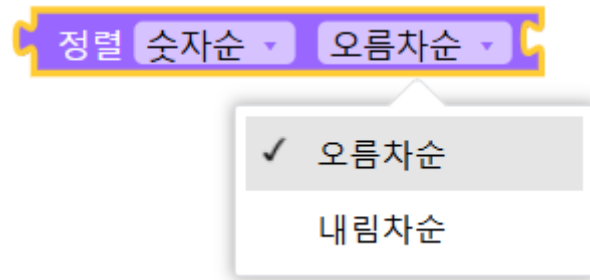


Figure 38: Image

Javascript 코드

```
function listsGetSortCompare(type, direction) { // 문자 비교 함수
  var compareFuncs = {
    'NUMERIC': function(a, b) {
      return Number(a) - Number(b); },
    'TEXT': function(a, b) {
      return String(a) > String(b) ? 1 : -1; },
    'IGNORE_CASE': function(a, b) {
      return String(a).toLowerCase() > String(b).toLowerCase() ? 1 : -1; },
  };
  var compare = compareFuncs[type];
  return function(a, b) { return compare(a, b) * direction; };
}

[].slice().sort(listsGetSortCompare("NUMERIC", 1)); // 숫자, 오름차순 정렬
[].slice().sort(listsGetSortCompare("NUMERIC", -1)); // 숫자, 내림차순 정렬
[].slice().sort(listsGetSortCompare("TEXT", 1)); // 문자, 오름차순 정렬
[].slice().sort(listsGetSortCompare("TEXT", -1)); // 문자, 내림차순 정렬
[].slice().sort(listsGetSortCompare("IGNORE_CASE", 1)); // 알파벳순(대소문자 구별 X), 오름차순 정렬
[].slice().sort(listsGetSortCompare("IGNORE_CASE", -1)); // 알파벳순(대소문자 구별 X), 내림차순 정렬
```

Python 코드

```
def lists_sort(my_list, type, reverse): # 정렬 함수
    def try_float(s):
        try:
            return float(s)
        except:
            return 0
    key_funcs = {
        "NUMERIC": try_float,
        "TEXT": str,
        "IGNORE_CASE": lambda s: str(s).lower()
    }
    key_func = key_funcs[type]
    list_cpy = list(my_list)
    return sorted(list_cpy, key=key_func, reverse=reverse)
```

lists_sort([], "NUMERIC", False) # 숫자, 오름차순 정렬

lists_sort([], "NUMERIC", True) # 숫자, 내림차순 정렬

lists_sort([], "TEXT", False) # 문자, 오름차순 정렬

lists_sort([], "TEXT", True) # 문자, 내림차순 정렬

lists_sort([], "IGNORE_CASE", False) # 알파벳순(대소문자 구별 X), 오름차순 정렬

lists_sort([], "IGNORE_CASE", True) # 알파벳순(대소문자 구별 X), 내림차순 정렬

아래 예제에서는 [gamma, beta, alpha] 리스트를 알파벳 오름차순으로 정렬하여 sortLetters 리스트를 생성하는 예제입니다.



Figure 39: Image

출력 결과는 [alpha, beta, gamma] 입니다.

JavaScript 코드

```
var letters, sortLetters;

function listsGetSortCompare(type, direction) { // 문자 비교 함수
    var compareFuncs = {
        'NUMERIC': function(a, b) {
            return Number(a) - Number(b); },
        'TEXT': function(a, b) {
            return String(a) > String(b) ? 1 : -1; },
        'IGNORE_CASE': function(a, b) {
            return String(a).toLowerCase() > String(b).toLowerCase() ? 1 : -1; },
    };
    var compare = compareFuncs[type];
    return function(a, b) { return compare(a, b) * direction; };
}

letters = ['gamma', 'beta', 'alpha'];
sortLetters = letters.slice().sort(listsGetSortCompare("TEXT", 1)); // 알파벳 순 오름차순 정렬
window.alert(sortLetters); // ['alpha', 'beta', 'gamma']
```

Python 코드

```
letters = None
sortLetters = None

def lists_sort(my_list, type, reverse): # 문자열 정렬 함수
    def try_float(s):
        try:
            return float(s)
        except:
            return 0
    key_funcs = {
        "NUMERIC": try_float,
        "TEXT": str,
        "IGNORE_CASE": lambda s: str(s).lower()
    }
    key_func = key_funcs[type]
    list_cpy = list(my_list)
    return sorted(list_cpy, key=key_func, reverse=reverse)
```

```
letters = ['gamma', 'beta', 'alpha']
sortLetters = lists_sort(letters, "TEXT", False) # 알파벳 순 오름차순 정렬
print(sortLetters) # ['alpha', 'beta', 'gamma']
```

뒤집기

리스트의 요소 순서를 역순으로 변경하는 블록입니다.



Figure 40: Image

Javascript 코드

```
[].slice().reverse(); // 리스트 역순으로 변경
```

Python 코드

```
list(reversed([])) # 리스트 역순으로 변경
```

아래 예제에서는 정렬된 sortLetters 리스트를 뒤집어 [gamma, beta, alpha] 리스트를 생성합니다.

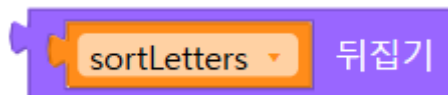


Figure 41: Image

Javascript 코드

```
var letters, sortLetters;

function listsGetSortCompare(type, direction) { // 문자 비교 함수
  var compareFuncs = {
    'NUMERIC': function(a, b) {
      return Number(a) - Number(b); },
    'TEXT': function(a, b) {
      return String(a) > String(b) ? 1 : -1; },
    'IGNORE_CASE': function(a, b) {
      return String(a).toLowerCase() > String(b).toLowerCase() ? 1 : -1; },
  };
  var compare = compareFuncs[type];
  return function(a, b) { return compare(a, b) * direction; };
}
```

```
}
```

```
letters = ['gamma', 'beta', 'alpha'];  
sortLetters = letters.slice().sort(listsGetSortCompare("TEXT", 1)); // 알파벳 기준 오름차순 정렬 ['alpha','beta','g  
sortLetters.slice().reverse(); // 리스트 역순으로 변경 ['gamma','beta','alpha']
```

Python 코드

```
letters = None  
sortLetters = None  
  
def lists_sort(my_list, type, reverse): # 문자열 정렬 함수  
    def try_float(s):  
        try:  
            return float(s)  
        except:  
            return 0  
    key_funcs = {  
        "NUMERIC": try_float,  
        "TEXT": str,  
        "IGNORE_CASE": lambda s: str(s).lower()  
    }  
    key_func = key_funcs[type]  
    list_cpy = list(my_list)  
    return sorted(list_cpy, key=key_func, reverse=reverse)  
  
letters = ['gamma', 'beta', 'alpha']  
sortLetters = lists_sort(letters, "TEXT", False) # 알파벳 기준 오름차순 정렬 ['alpha','beta','gamma']  
list(reversed(sortLetters)) # 리스트 역순으로 변경 ['gamma','beta','alpha']
```

우리는 변수라는 용어를 수학이나 다른 프로그래밍 언어에서 사용되는 것과 동일한 의미로 사용합니다. 즉, 값을 저장하며 변경할 수 있는 이름을 가진 요소를 의미합니다. 변수는 여러 가지 방법으로 생성할 수 있습니다.

- ☐ **으로 계산 및 각 항목에 대해**와 같은 일부 블록은 변수를 사용하며, 해당 변수의 값을 정의합니다. 이러한 변수는 전통적인 컴퓨터 과학 용어로 **반복 변수(loop variables)** 라고 합니다.
- ☐ **사용자 정의 함수** 는 입력값을 정의할 수 있으며, 이는 함수 내에서만 사용할 수 있는 변수(매개변수 또는 인자)를 생성합니다.
- ☐ 사용자는 **설정** 블록을 통해 언제든지 변수를 생성할 수 있습니다. 이는 전통적으로 “전역

변수(global variables)” 라고 불립니다.

☐ 블록 컴포저는 지역 변수(local variables) 를 지원하지 않습니다.

드롭다운 메뉴

변수의 드롭다운 기호(삼각형)를 클릭하면 다음과 같은 메뉴가 나타납니다.

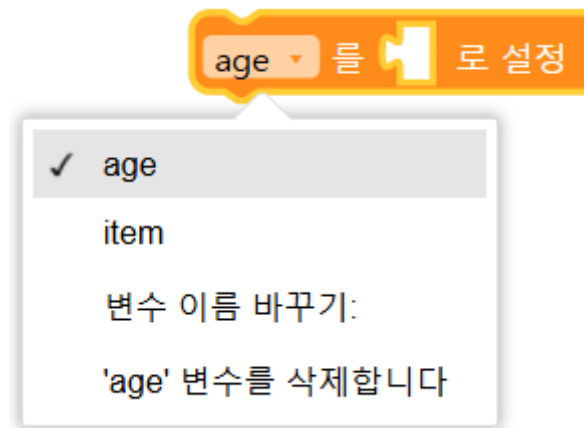


Figure 42: Image

이 메뉴에서 다음 옵션을 선택할 수 있습니다.

☐ 프로그램에서 정의된 모든 기존 변수 이름이 표시됩니다.

☐ “변수 이름 바꾸기”: 프로그램 전체에서 해당 변수의 이름을 변경합니다. 이 옵션을 선택하면 새 이름을 입력할 수 있는 창이 나타납니다.

☐ “변수 삭제”: 프로그램에서 이 변수를 참조하는 모든 블록을 삭제합니다.

블록

설정

설정 블록은 변수에 값을 할당하며, 해당 변수가 존재하지 않을 경우 새 변수를 생성합니다. 예를 들어, 아래 블록은 "age"라는 변수를 만들고 값 12를 할당합니다.



Figure 43: Image

Javascript 코드


```
var age;  
age = 12; // age 변수에 12 할당
```

Python 코드

```
age = None  
age = 12 # age 변수에 12 할당
```

값 가져오기

아래 블록은 변수에 저장된 값을 제공하지만, 해당 값을 변경하지는 않습니다.

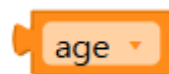


Figure 44: 이미지

설정 블록 없이 블록을 사용하는 것도 가능하지만, 이는 올바른 프로그래밍 방식이 아닙니다.

Javascript 코드

```
age; // 변수 값 가져오기
```

Python 코드

```
age # 변수 값 가져오기
```

바꾸기

바꾸기 블록은 변수의 값에 숫자를 더하는 역할을 합니다.



Figure 45: Image

위 블록은 다음 코드와 동일한 기능을 수행하는 단축 표현입니다.

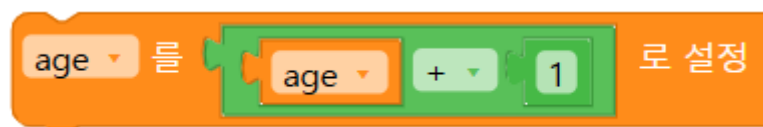


Figure 46: Image

Javascript 코드

```
var age;  
age = (typeof age === 'number' ? age : 0) + 1; // age 1만큼 더하기
```

Python 코드

```
from numbers import Number  
  
age = None  
age = (age if isinstance(age, Number) else 0) + 1 # age 1만큼 더하기
```

예제

다음은 변수 사용 예제입니다.



Figure 47: Image

첫 번째 줄의 블록은 "age"라는 변수를 만들고 초기 값을 숫자 12로 설정합니다. 두 번째 줄의 블록은 값 12를 가져와서 1을 더한 후, 그 합(13)을 변수에 저장합니다. 마지막 줄은 다음 메시지를 표시합니다: "13살 생일축하해!"

Javascript 코드

```
var age;  
  
age = 12;  
age = (typeof age === 'number' ? age : 0) + 1; // age 1만큼 더하기  
window.alert(String(age) + '살 생일축하해!');
```

Python 코드

```
from numbers import Number  
  
age = None
```

```
age = 12
age = (age if isinstance(age, Number) else 0) + 1 # age 1만큼 더하기
print(str(age) + '살 생일축하해!')
```